“Tab, Tab, Bug”

Security Pitfalls of Next Edit Suggestions in AI-Integrated IDEs

Yunlong Lyu* Yixuan Tang* Peng Chen Tian Dong
Xinyu Wang Zhiqiang Dong Hao Chen

The University of Hong Kong McGill University Independent Researchers

NES security study

*Equal contribution.

The talk at a glance

Understand the mechanism

1. What is NES?
2. Problem statement and motivation
3. Related work and the gap
4. Taxonomy

Security Evaluation

5. White-box evaluation
6. Black-box evaluation
7. User study
8. Results and findings
9. Conclusions

NES predicts the next edit, including where it belongs

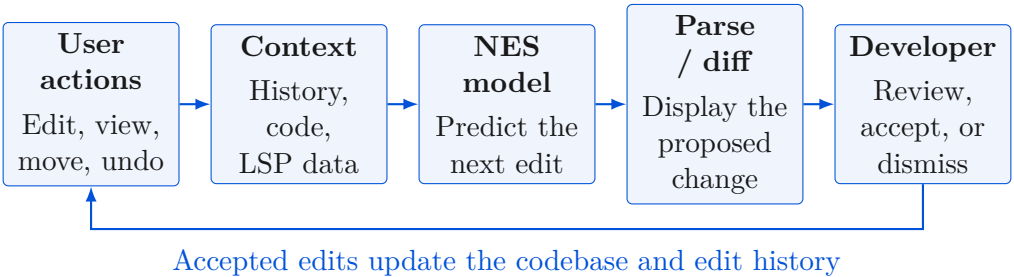
Next Edit Suggestions (NES)

An AI-integrated IDE continuously uses recent interactions and project context to infer the developer's intent and proactively suggest the next code modification.

	Conventional completion	NES
Primary signal	Local code near the cursor	Recent edits and broader project context
Suggested change	Continue code at the cursor	Insert, delete, or replace code across lines or files
Interaction	Complete the current line	Review, navigate, accept; repeat with Tab

Source: supplied paper, Sections 1–2.

Inside the context-to-edit pipeline



Takeaway

The security boundary spans **triggers, retrieved context, predicted edits, and acceptance**.

Source: supplied paper, Section 2, Figure 1; simplified architectural redraw.

Productive editing can silently expand security exposure

1. **Implicit retrieval:** secrets or unsafe patterns enter the model's context through viewed code, history, or dependencies.
2. **Semantic changes:** a refactor can preserve functionality while losing authorization, confidentiality, or parser protections.
3. **Low-friction acceptance:** multi-location suggestions make complete security review harder during a Tab-by-Tab flow.

Takeaway

Assessing the generated snippet alone misses risks created by the IDE workflow.

Source: supplied paper, Sections 1–2.

Prior work establishes insecurity, poisoning, and human factors

- **Generated-code security**
  - Pearce et al. (2022); Khoury et al. (2023): insecure implementations and defaults.
- **Developer interaction**
  - Perry et al. (2023); Sandoval et al. (2023): trust, prompting, and verification affect security.
- **Adversarial manipulation**
  - Schuster et al. (2021); Oh et al. (2024): poisoning can steer assistants toward unsafe output.

Gap: NES implicitly collects an ongoing edit stream and proactively proposes changes. This study connects these mechanisms to security failures.

Source: supplied paper, Sections 1 and 7; references [27, 37, 43–46].

Taxonomy: context contamination (V1–V6)

	Risk vector	Failure manifestation
V1	Pre-trained model	Suggest weak cryptographic primitives without a contaminated input.
V2	Recently viewed code	Reproduce a secret from a viewed configuration file.
V3	Edit history	Copy unsafe SQL concatenation from recent edits.
V4	Undo operation	Reintroduce a credential the developer removed.
V5	Structural context	Use an unsafe API without preserving local security constraints.
V6	Cross-file dependencies	Carry insecure testing patterns into production code.

Source: supplied paper, Table 1; Section 3.1.1.

Taxonomy: transactional edits and human–IDE interaction

	Risk vector	**Failure manifestation**
V7	Variable logging	Copy a logging pattern onto sensitive variables.
V8	Visibility refactoring	Expose internal sensitive methods as endpoints.
V9	Component replacement	Omit parser-specific XXE protections after migration.
V10	Location jumping	Skip intermediate vulnerable sites during remediation.
V11	No-Op edit	Stop suggesting before the entire fix is complete.
V12	Sequential edits	Introduce an insecure suggestion after a correct streak.

Source: supplied paper, Table 1; Sections 3.1.2–3.1.3.

White-box design: control the context, inspect the resulting edit

1. Scan the top **1,000 Java-only GitHub projects** with CodeQL for representative insecure patterns.
2. Build a safe but incomplete target at the cursor; retain surrounding artifacts as potential propagation sources.
3. Reconstruct NES prompts using **tree-sitter**, structural context, dependencies, and synthetic edit history.
4. Query **Zeta**, apply the suggested edit, and check whether the targeted risk manifests.

Scale: 410 base cases across 9 CWE categories; 10 repetitions per case, yielding 4,100 runs.

Source: supplied paper, Sections 3.1 and 4; Table 2.

Security checking combines structural and semantic evidence

- **AST checks:** vulnerable calls or missing configuration.
- **Regex checks:** secret literals and unsafe lexical patterns.
- **Semantic judges:** sensitivity, data flow, or edit intent that syntax cannot resolve.

LLM adjudication

Five independent judges.

At least **4 of 5** must flag a suggestion as insecure.

51 split-vote cases receive manual review.

Takeaway

These tests show when NES makes unsafe edits, not how often this happens in everyday coding.

Source: supplied paper, Section 3.1; Appendices D–E.

Black-box design: reproduce the workflow in four IDEs

Systems under test

Cursor
GitHub Copilot
Trae
Zed

10 cases per vector
$12 \times 10 = 120$ selected cases
$120 \times 4 = 480$ executions

Manual protocol

1. Open the project at the target location.
2. Recreate preparatory viewing and editing actions.
3. Trigger NES and inspect the suggestion.
4. Reload the window before the next case.

Source: supplied paper, Section 3.2; Appendix H.

User study: a survey of experience, trust, and scrutiny

Recruitment and filtering

- One multinational IT company and one U.S. university.
- 385 responses collected; 241 professionals in the initial pool.
- **269 retained responses** after attention-check filtering.

Questionnaire

- 12 questions covering demographics, NES usage, trust, and security perceptions.
- Consent, anonymity, compensation, and IRB approval reported.
- Retained cohort: 64.4% professional; 56.9% use NES often or always.

Source: supplied paper, Sections 5.1–5.2; Appendices B and G.

Reported headline rates show failures across both settings

74.44%

White-box: Zeta

Macro-average across 12 vectors

77.92%

Black-box: four IDEs

Average across vectors and IDEs

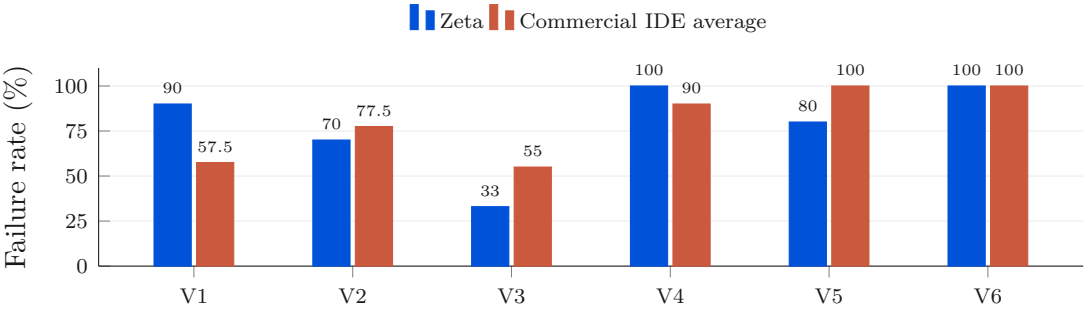
Takeaway

Both Zeta and commercial IDEs frequently produce unsafe edits in the tested security scenarios.

These are averages across risk vectors. White-box: 4,100 runs; black-box: 480 executions.

Source: supplied paper, Section 4; Table 2.

C1 results: expanded context carries unsafe patterns

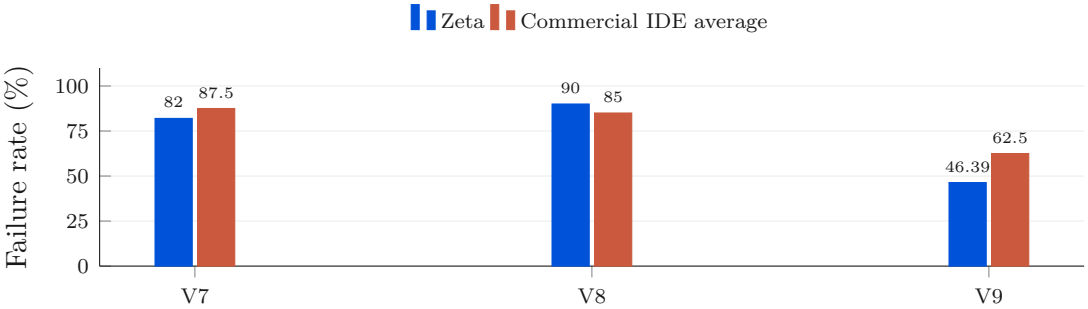


V1 model; V2 viewed code; V3 history; V4 undo; V5 structure; V6 dependencies.

Finding: undo may remove a secret from the file while leaving it in model context.

Source: supplied paper, Table 2; Sections 4.1.1 and 4.2.

C2 results: functional refactoring loses security invariants

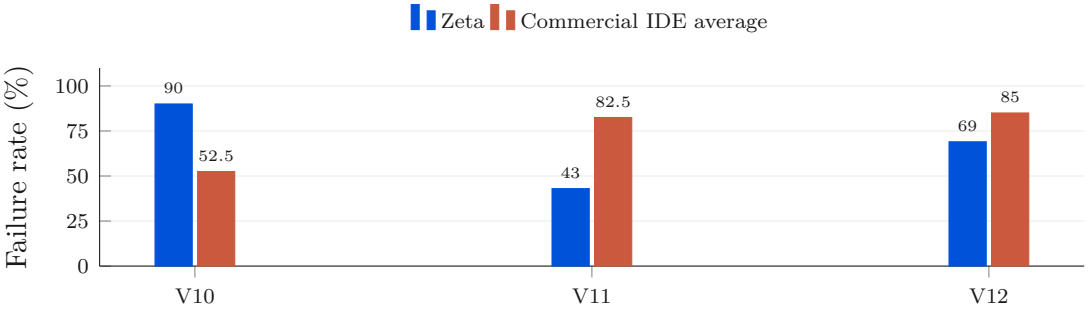


V7: sensitive logging; V8: endpoint exposure; V9: missing XML-parser protections.

Finding: functionally equivalent replacements can lose security settings.

Source: supplied paper, Table 2; Sections 4.1.2 and 4.2.

C3 results: jumps, silence, and insecure later steps



V10: location jumping; V11: No-Op edit; V12: sequential edits.

These are simulated workflow failures, not measured human acceptance probabilities.

Source: supplied paper, Table 2; Sections 3.1.3 and 4.1.3–4.2.

Survey findings: exposure exceeds explicit security checking

81.1%
Report encountering insecure NES suggestions

62.4%
Report test-to-production data suggestions

12.3%
Explicitly check suggestion security

- 28.6% report always thoroughly reviewing NES output.
- 32.0% report skimming or rarely reviewing it.
- 55.0% select clean-looking code as a reason for acceptance.

Source: supplied paper, Sections 5.4–5.5; Figures 5–6; retained $N = 269$.

What limits the interpretation of these findings?

- **Technical scope:** Java, one open NES model, and four commercial IDEs at study time.
- **Sampling:** scenarios intentionally activate risks; black-box testing uses ten cases per vector per IDE.
- **Measurement:** reconstructed prompts and hybrid detectors approximate production behavior; proprietary internals remain hidden.
- **Human evidence:** self-reports from one company and one university; no causal acceptance experiment or population-wide estimate.

Takeaway

The study demonstrates mechanism-linked failures; everyday prevalence and mitigation effectiveness remain open questions.

Source: supplied paper, Sections 3, 5, and 6.

Security must follow the entire context-to-edit workflow

For IDE providers

- Make context sources and exclusions visible.
- Filter suspicious context and suggestions.
- Flag security-sensitive changes before acceptance.
- Evaluate multi-step edits and incomplete remediation.

For developers and teams

- Treat suggested edits as unverified code.
- Review the full transaction and skipped sites.
- Check secrets, authorization, and migration settings explicitly.
- Combine review, security analysis, and secure-coding training.

Source: supplied paper, Section 6; transaction-level review follows the reported failure mechanisms.

Three messages to take away

1. **NES broadens the security boundary.** Ambient interactions and project context shape proactive edits.
2. **Better base models do not resolve every workflow risk.** Context propagation and lost security invariants appear across the tested systems.
3. **Review must cover the sequence.** Fluent code, navigation, or silence does not establish secure completion.

A Tab press accepts an edit;
it does not establish its security.

Source: supplied paper, Sections 4–8.